

1. Introduction

The Min-Max Array Program is a C program designed to accept an array of integer values from the user and determine the minimum and maximum values present in the array. The program also contains four test cases to verify different conditions related to the input array and the calculated maximum and minimum values.

The program is divided into three source files:

- `main.c` – Contains the main program, user input, test-case selection, and test execution.
- `fout.c` – Contains the implementation of the `findminmax()` function.
- `header.h` – Contains the declarations and required header files used by the program.

The program uses the `assert()` function to perform basic checks during testing.

2. Objective

The main objectives of the program are:

1. To accept the size of an integer array from the user.
2. To accept the elements of the array from the user.
3. To find the minimum and maximum values in the array.
4. To display the maximum value obtained from the array.
5. To provide test cases for checking different array conditions.
6. To use assertions for validating the expected conditions.

3. Program Structure

The program consists of three files:

3.1 `main.c`

The `main.c` file contains the `main()` function. It is responsible for:

- Taking the array size as input.
- Creating the integer array.
- Reading array elements from the user.
- Calling the `findminmax()` function.
- Displaying the maximum value.
- Allowing the user to select and execute test cases.

The program includes `header.h` at the beginning so that the function declarations and required definitions can be accessed.

3.2 `fout.c`

The `fout.c` file contains the implementation of the `findminmax()` function.

The function receives:

- The input array.
- The size of the array.
- A pointer to store the maximum value.
- A pointer to store the minimum value.

The function processes the array and determines its minimum and maximum values. The results are returned to `main()` using pointer variables.

3.3 `header.h`

The `header.h` file acts as the header file for the project. It contains the declarations required by the source files.

The `findminmax()` function prototype is declared in this file so that it can be called from `main.c`.

The required standard libraries, such as the library used for assertions, can also be included through the header file.

4. Working of the Program

The execution of the program begins from the `main()` function.

First, the program asks the user to enter the size of the array.

For example:

enter array size:5

An integer array of the specified size is then created.

The program asks the user to enter each element of the array:

element 1 1

element 1 2

element 1 3

...

After all elements are entered, the `findminmax()` function is called.

```
findminmax(usrarr,n,&max,&min);
```

The addresses of `max` and `min` are passed to the function. This allows the function to modify these variables and return the calculated minimum and maximum values to the `main()` function.

The maximum value is then displayed:

max is <value>

After this, the program allows the user to select a test case.

5. Test Cases

The program contains four test cases. Each test case checks a different condition.

Test Case 1 – Multiple Occurrences of Maximum

The first test case checks whether the maximum value occurs at least twice in the input array.

The program iterates through the array and compares each element with the calculated maximum value.

The expected condition is:

Number of occurrences of maximum ≥ 2

If the assertion condition is satisfied, the test case is considered passed.

Expected Result

The test should pass when the maximum value occurs two or more times in the input array.

Output Screenshot

```
msis@msis-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Abhay_dls/dsa_lab/min_max$ ./a.out
enter array size:4
element 1 1 1
element 1 2 2
element 1 3 2
element 1 4 4
max is 4
select test case to run:(1),(2),(3),(4)
1
passed
```

Test Case 2 – Uniform Array

The second test case is intended to check a uniform array, where all elements have the same value.

For example:

Input array: 5 5 5 5 5

For a uniform array:

Minimum = Maximum = 5

Therefore, the expected condition is that the elements of the array are equal to both the minimum and maximum values.

Expected Result

The test should confirm that all elements in a uniform array have the same value.

Output Screenshot

```
msis@msis-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Abhay_dls/dsa_lab/min_max$ ./a.out
enter array size:1
element 1 1 1
max is 1
select test case to run:(1),(2),(3),(4)
2
Uni array
```

Test Case 3 – Identical Elements

The third test case checks whether all elements in the array are identical.

For example:

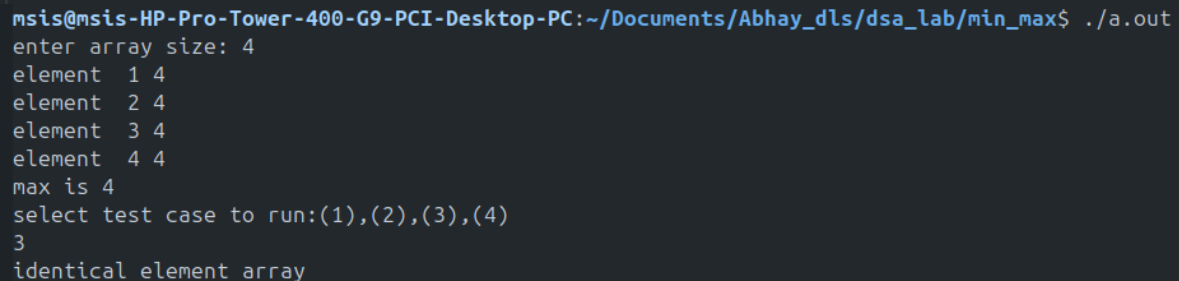
Input array: 7 7 7 7

Since every element has the same value, the array can be considered an identical-element array.

Expected Result

The test should pass when every element in the array has the same value as the other elements.

Output Screenshot



```
msis@msis-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Abhay_dls/dsa_lab/min_max$ ./a.out
enter array size: 4
element 1 4
element 2 4
element 3 4
element 4 4
max is 4
select test case to run:(1),(2),(3),(4)
3
identical element array
```

Test Case 4 – Array Not NULL

The fourth test case checks that the array pointer is not **NULL**.

The test uses an assertion to verify the pointer condition.

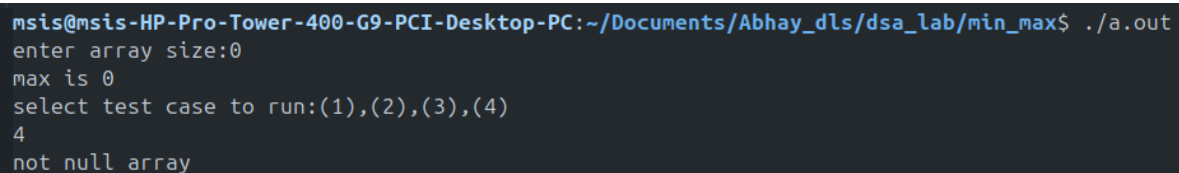
```
assert(usrarr != NULL);
```

Since the array is created inside the `main()` function, the program expects the array reference to be valid during its execution.

Expected Result

The assertion should pass when the array reference is valid.

Output Screenshot

A terminal window screenshot showing the execution of a program. The prompt is 'msis@msis-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Abhay_dls/dsa_lab/min_max\$'. The user enters './a.out'. The program prompts 'enter array size:0', then 'max is 0', then 'select test case to run:(1),(2),(3),(4)'. The user enters '4', and the program outputs 'not null array'.

```
msis@msis-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Abhay_dls/dsa_lab/min_max$ ./a.out
enter array size:0
max is 0
select test case to run:(1),(2),(3),(4)
4
not null array
```

6. Use of Assertions

The program uses the `assert()` function to perform checks during testing.

An assertion evaluates a condition. If the condition is true, execution continues. If the condition is false, the program terminates and reports the failed assertion.

For example:

```
assert(condition);
```

Assertions are useful during development and testing because they help identify conditions that do not behave as expected.

In this program, assertions are used in the test cases to verify conditions related to the maximum value, minimum value, array elements, and array validity.

9. Conclusion

The Min-Max Array Program demonstrates the use of arrays, functions, pointers, conditional statements, loops, and assertions in C.